University of Nottingham
UK | CHINA | MALAYSIA

COMP4139
Machine Learning (MLE)

Sequence Models (Transformers)

Direnc Pekaslan
Shreyank Narayana Gowda

Assistant Professor
School of Computer Science
University of Nottingham

University of Nottingham
UK | CHINA | MALAYSIA

Padlet -- General Questions for the whole module ☺

Learning Objective

- Understand Transformers architecture
  - Encoder-decoder mechanism
  - Apply positional encoding concepts
  - Attention and multi-head attention
  - Outline training and inference steps

Transformers

- LSTM is able to capture long term memory but not long enough.
- Transformers is able to capture dependencies between each input words and between all input words to outputs in a sentence.

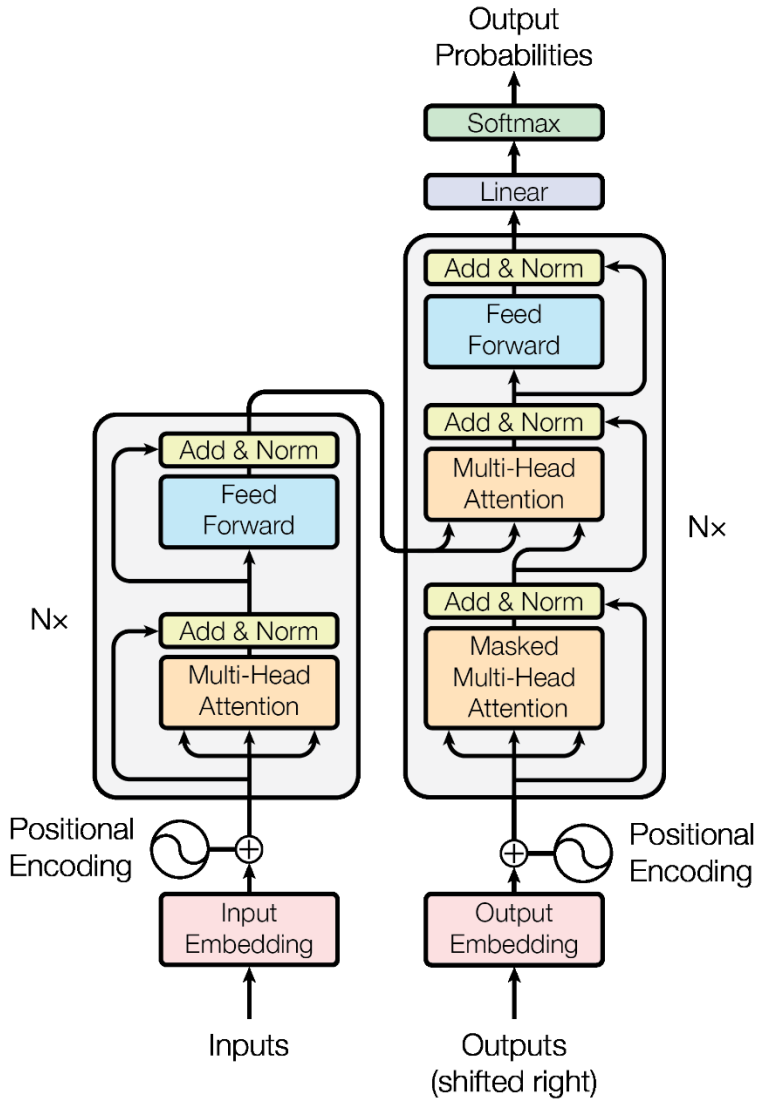
Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

https://arxiv.org/abs/1706.03762

Overview of Transformers

- Encoder-decoder architecture
- The Encoder output is a continues vector representation of the inputs
- The Decoder takes this continues vector representation to generate output one by one.
- Applications: language translation, chatbot, etc.
- Transformers has been used everywhere (e.g. object classification, image segmentation, etc.)



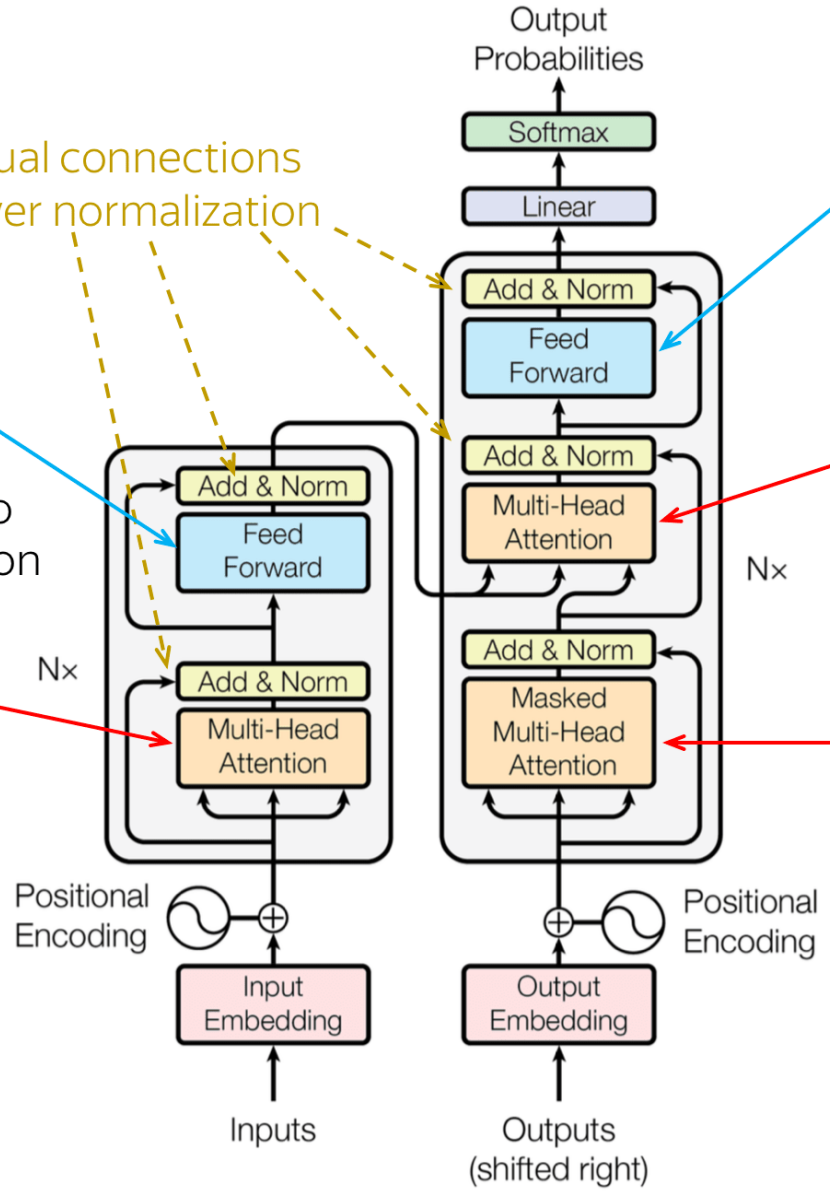
Overview of Transformers

Residual connections and layer normalization

Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values are computed from encoder states



Feed-forward network:
after taking information from other tokens, take a moment to think and process this information

Decoder-encoder attention:
target token looks at the source

queries – from decoder states; keys and values from encoder states

Decoder self-attention (masked):
tokens look at the previous tokens

queries, keys, values are computed from decoder states

Input Embedding

- Each word is mapped to a vector (37K)

$$\text{how} = \begin{bmatrix} 0.2 \\ 0.3 \\ 0.7 \\ 0.5 \end{bmatrix} \quad \text{are} = \begin{bmatrix} 0.5 \\ 0.1 \\ 0.4 \\ 0.3 \end{bmatrix} \quad \text{you} = \begin{bmatrix} 0.6 \\ 0.2 \\ 0.9 \\ 0.3 \end{bmatrix}$$

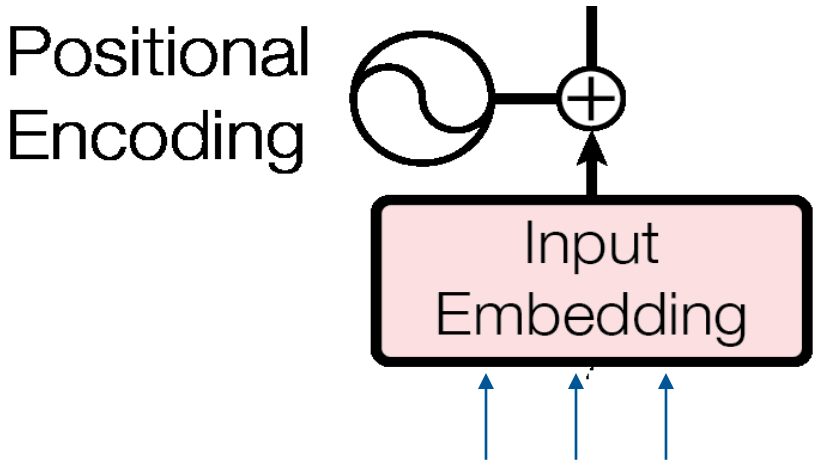
i=0, i=1, i=2, i=3

P=0 P=1 P=2

- Add a positional encoding to the vector using sine/cosine functions

$$PE_{p,i} = \begin{cases} \sin\left(\left(\frac{p}{10000}\right)^{i/d}\right) & i \text{ even} \\ \cos\left(\left(\frac{p}{10000}\right)^{\frac{(i-1)}{d}}\right) & i \text{ odd} \end{cases}$$

i=0 i=1 i=2 i=3

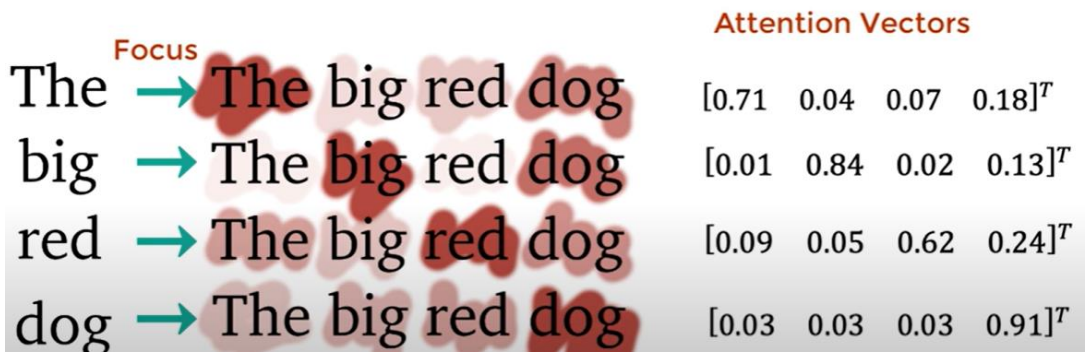


P → word index

i → Feature value index

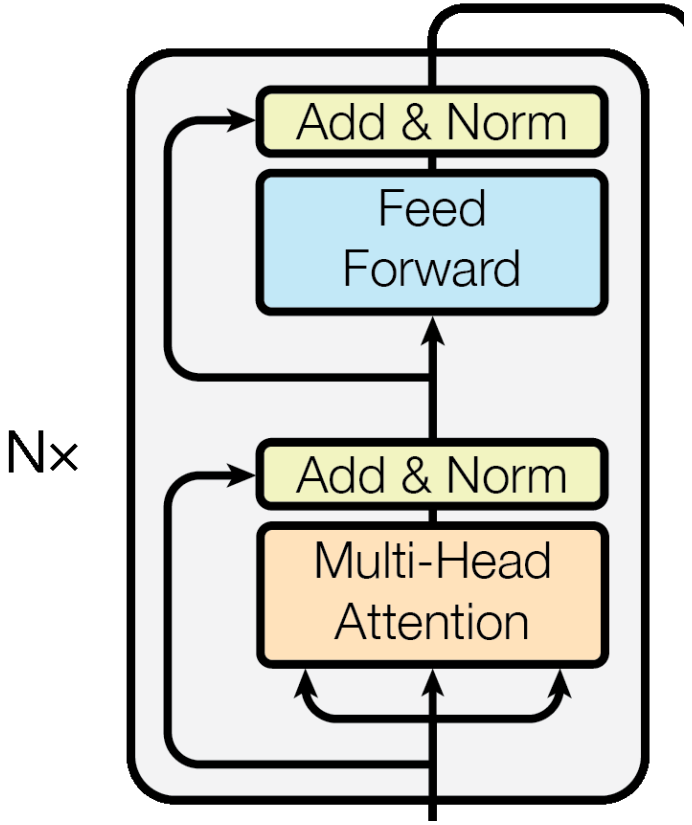
Encoder Training

- The multi-head attention calculates the relationships between all pairwise inputs



- The feed forward network (MLP) maps the attention vectors to something can be feed to the Transformer decoder.

Vector representation of inputs



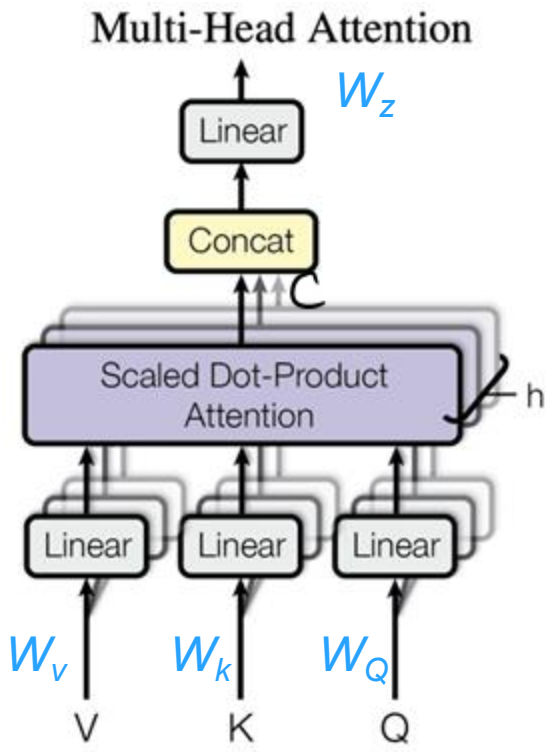
Multi-Head Attention

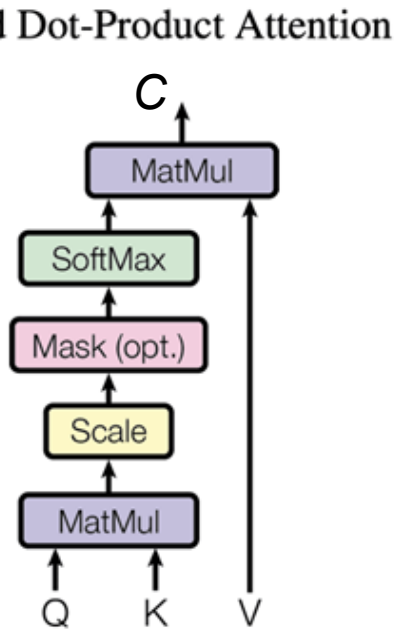
- Input, each word has Query, Key and Value (same copy)

How	are	you
0.2	0.5	0.6
0.3	0.1	0.2
0.7	0.4	0.9
0.5	0.3	0.3

- Scaled Dot-Product Attention->Scale->Softmax

	How	**are**	**you**
How	0.7	0.2	0.1
are	0.2	0.6	0.2
you	0.1	0.2	0.7





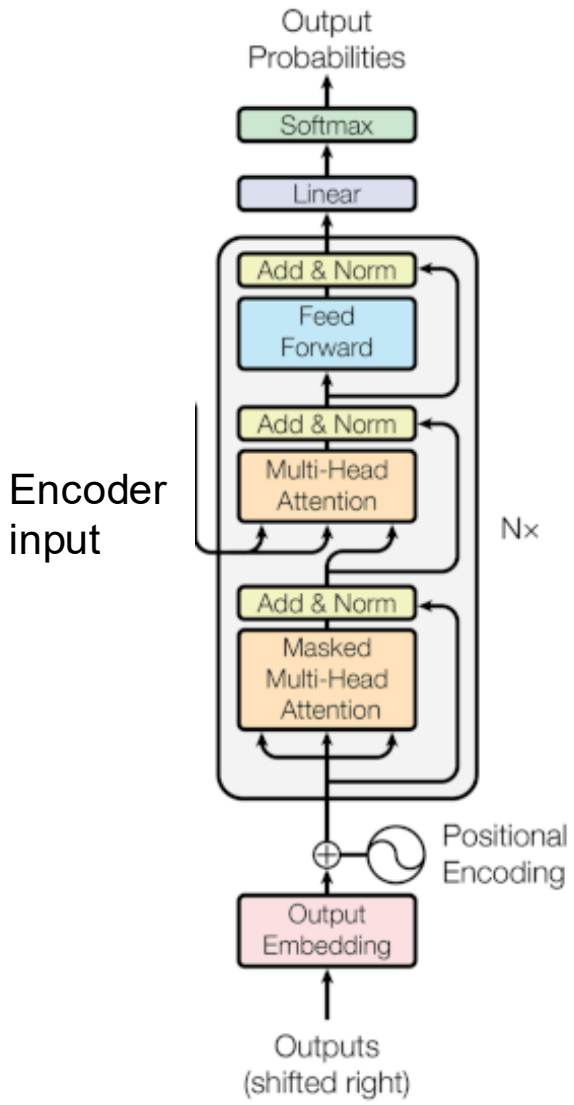
- For each input (e.g. word) compute multiple attention vectors (learnable weights W_V, W_k, W_Q) and use the weighted (W_z) average as the final attention vector for each word

$$\mathrm{C} = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

- Output vector (C) is the dot product between attention weight and value. Multiple words are processed in parallel.

Decoder Training

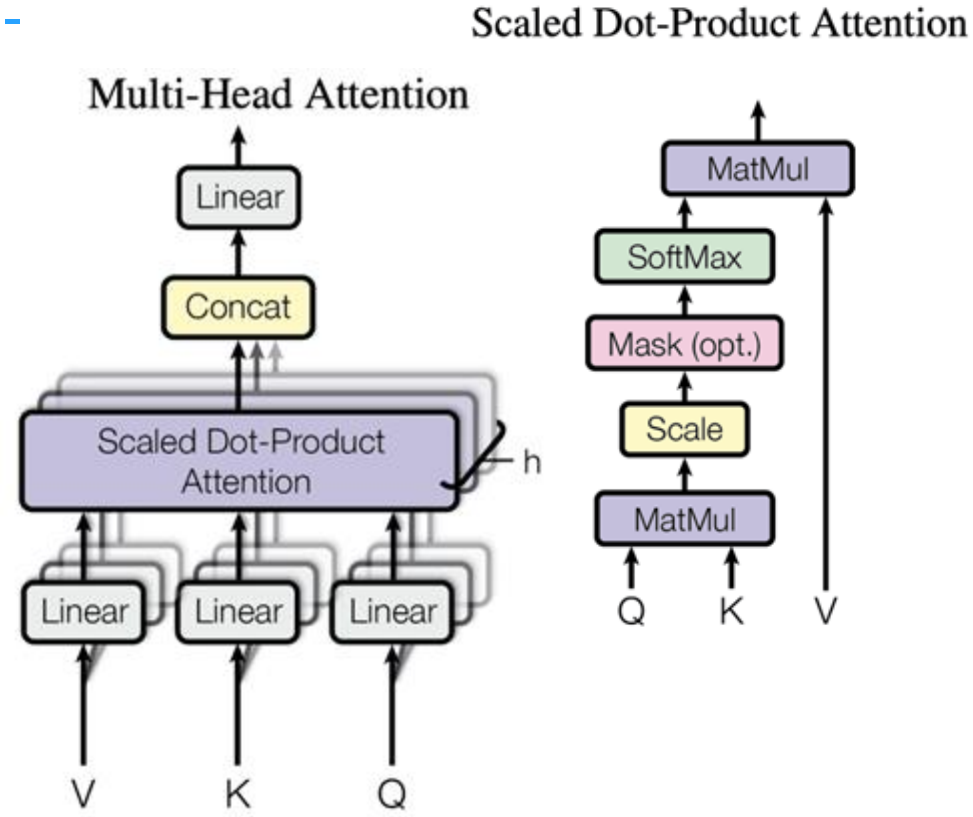
- During training: the target sentence is input to the Masked Multi-head attention. Masked out relationships to future words



Decoder Training

- During training: the target sentence is input to the Masked Multi-head attention. Masked out relationships to future words

	start	I	am	fine
start	1	0	0	0
I	0	0.7	0	0
am	0	0.2	0.6	0
fine	0	0.1	0.2	0.7



$$\mathrm{C} = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Decoder Training

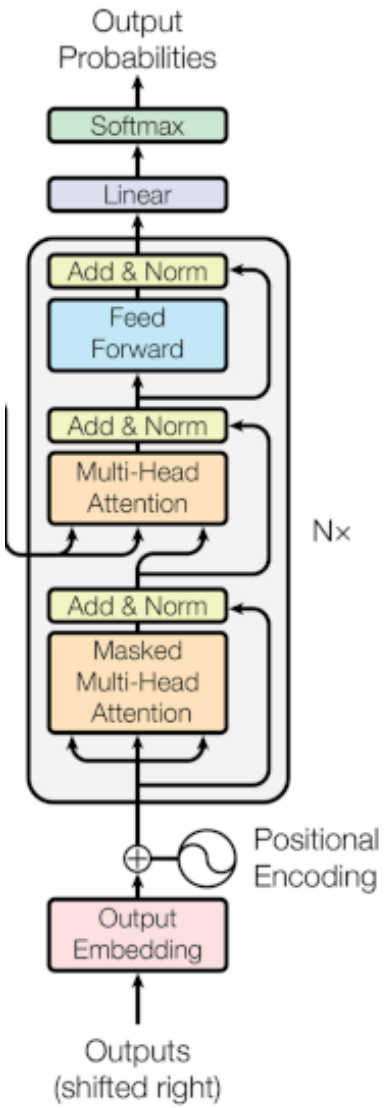
- During training: the target sentence is input to the Masked Multi-head attention. Masked out relationships to future words

	start	I	am	fine
start	1	0	0	0
I	0	0.7	0	0
am	0	0.2	0.6	0
fine	0	0.1	0.2	0.7

- Then another Multi-head attention that learns the interactions between input words and target words (key and value are from the encoder, query is from decoder)

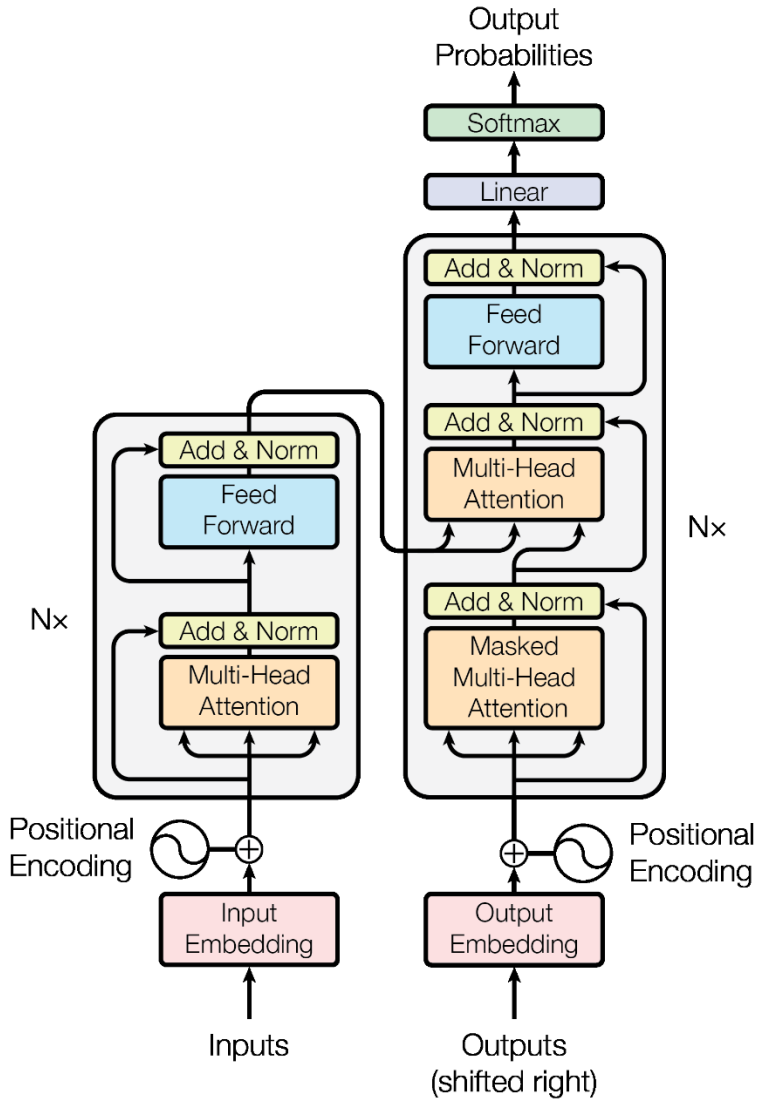
	how	are	you
I	0.6	0.3	0.1
am	0.25	0.7	0.05
fine	0.15	0.05	0.8

- The output is one-hot encoding (e.g. 1000-word vector)



Model Inference

- The input sentence is input to the encoder to generate feature vector representation and feed to the decoder
- A "start" input signal is input to the decoder
- The decoder generate the first possible word in the target sentence.
- This estimated word is then input the decoder to estimate the next word and iterate until an "end" signal.
- To estimate the second words onwards the model uses the whole input sentence and all previously generated target words



Popular Transformer Methods

- Attention block is the key element in Transformers and can be flexibly used in many applications
- Transformer models normally requires a lot of data to train. Pre-trained model is normally used (i.e. transfer learning)
- BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". https://arxiv.org/abs/1810.04805
- An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale: https://arxiv.org/abs/2010.11929
- GPT-3: Understand and generate natural language (e.g. chatbot) https://beta.openai.com/docs/models/gpt-3

Chat GPT

Three main steps in training Chat GPT:

1. Generative pre-training.
   Solution: Transformer trained on large number of texts.
   Problem: Generated texts do not necessarily answer the question (prompt).

2. Supervised fine-tuning.
   Solution: Human trainers to respond to prompt that are used as the label.
   Problem: Machine responses are not matched exactly to human's response with some false/forbidden/non-sense statements.

3. Reinforcement learning from human feedback.
   Solution: human trainers rank the responses and form a reward function to make machine behave. (proximal policy optimisation).

Training language models to follow instructions with human feedback. https://arxiv.org/abs/2203.02155

Visuals

https://poloclub.github.io/transformer-explainer/

https://bbycroft.net/llm

Learning Objective

- Understand Transformer architecture
  - Encoder-decoder mechanism
  - Apply positional encoding concepts
  - Attention and multi-head attention
  - Outline training and inference steps